

Pet Territory Wars — MVP Database Design v1.0

Doküman sürümü: 1.0.0

Ana veritabanı: PostgreSQL + PostGIS

Backend: Go

Mobil istemci: Flutter

Mimari: Modüler monolith

Kapsam: Faz 0 Backend Simulation + Faz 1 Barebone Mobile

Öncelikler: Veri doğruluğu, transaction güvenliği, yüksek eşzamanlılık, sorgu performansı ve kontrollü ölçeklenme

1. Amaç

Bu doküman Pet Territory Wars MVP'sinin kalıcı veri modelini tanımlar.

Database'in temel sorumlulukları:

- Oyuncu ve pet bilgilerini saklamak
- Walk lifecycle ve idempotency bilgisini korumak
- Raw GPS rotalarını güvenli biçimde saklamak
- Territory ownership ve dominance state'ini authoritative olarak tutmak
- Aynı hex üzerindeki eşzamanlı güncellemeleri kontrol etmek
- Haftalık ve lifetime skorları saklamak
- RuleSet sürümlerini değişmez biçimde saklamak
- Domain event'lerini outbox üzerinden güvenli biçimde taşımak
- Harita ve leaderboard sorgularını ölçeklenebilir biçimde desteklemek

Database'in amacı yalnızca veri saklamak değildir.

Oyunun authoritative state'ini tutarlı, açıklanabilir ve geri alınabilir biçimde korumaktır.

2. Teknoloji Kararı

2.1 Ana Database

PostgreSQL
+
PostGIS

PostGIS; şehir sınırları, polygon kontrolleri, viewport sorguları ve diğer mekânsal ihtiyaçlar için kullanılacaktır. PostGIS, PostgreSQL üzerinde geometry/geography nesnelerini ve GiST tabanlı spatial index desteğini sağlar.

H3, oyun grid sistemidir.

PostGIS ise:

- Şehir polygon'ları
- Point-in-polygon
- Bounding box
- Viewport
- Geometrik doğrulama

için kullanılacaktır.

Bu iki sistem birbirinin alternatifi değildir.

H3 → oyun alanı ve ownership kimliği
PostGIS → gerçek dünya geometrisi ve spatial sorgular

2.2 MongoDB Kararı

MVP'de MongoDB kullanılmayacaktır.

İlk sürümde:

PostgreSQL → authoritative state
Redis → yalnızca cache / kısa süreli coordination
Object storage → ileride raw route arşivi

yeterlidir.

MongoDB'nin potansiyel kullanım alanları:

- Büyük document tabanlı telemetry verileri
- Esnek analytics event'leri
- Şeması çok hızlı değişen bağımsız log verileri
- Ayrı bir read workload

Ancak bunlar MVP'nin ana database ihtiyacı değildir.

MongoDB eklenirse:

- İki farklı backup sistemi
- İki farklı migration yaklaşımı
- İki farklı monitoring sistemi
- Cross-database consistency problemi
- İki farklı transaction sınırı
- Daha fazla operasyonel maliyet

oluşur.

MongoDB'nin embedding yaklaşımı bazı read operasyonlarını azaltabilir; ancak çok sayıda karmaşık ilişki ve sık güncellenen ortak state için referans ve transaction ihtiyacı yeniden ortaya çıkar. MongoDB dokümantasyonu da embedding'in karmaşık many-to-many ilişkiler için uygun olmayabileceğini belirtir.

Sonuç:

Ölçülmüş bir darboğaz ortaya çıkmadan ikinci database eklenmeyecektir.

3. Database Tasarım Prensipleri

3.1 Tek Source of Truth

Her veri için tek authoritative kaynak olmalıdır.

Hex owner	→ territory_hexes
Stored dominance	→ territory_hexes
Walk lifecycle	→ walks
Raw route	→ walk_points
Weekly score	→ player_season_scores
Lifetime score	→ player_lifetime_stats
Rules	→ rule_sets
Event delivery	→ outbox_events

Aynı authoritative bilgi iki tabloda bağımsız şekilde tutulmamalıdır.

3.2 Kontrollü Denormalizasyon

Gereksiz kopya veri tutulmayacaktır.

Ancak performans için bilinçli derived state kullanılabilir.

Örnek:

```
territory_hexes.owner_id
```

authoritative ownership bilgisidir.

```
player_season_scores.active_hex_count
```

ise hızlı leaderboard sorgusu için denormalize edilmiş sayaçtır.

Bu sayaç:

- Source of truth değildir.
- Territory tablosundan yeniden oluşturulabilir.
- Atomic delta ile güncellenir.
- Düzenli reconciliation ile kontrol edilebilir.

3.3 Foreign Key Kullanımı

Core ilişkiler foreign key ile korunacaktır.

Foreign key kullanılacak temel ilişkiler:

```
pets.owner_id → players.id
walks.player_id → players.id
walks.pet_id → pets.id
walks.city_id → cities.id
walks.season_id → seasons.id
walks.rule_set_id → rule_sets.id
territory_hexes.city_id → cities.id
territory_hexes.owner_id → players.id
player_season_scores.player_id → players.id
player_season_scores.season_id → seasons.id
```

Ancak yüksek hacimli append-only log ve outbox tablolarında ilişki maliyeti ayrıca değerlendirilir.

3.4 UUID ve Dahili Kimlikler

Public nesneler için UUID kullanılacaktır.

Öneri:

UUIDv7

avantajları:

- Dağıtık olarak üretilebilir
- Tahmin edilmesi zordur
- Zamana yakın sıralı olduğu için rastgele UUIDv4'e göre index locality daha iyidir

Ancak PostgreSQL'in kurulum ve kütüphane desteğine göre UUID üretimi application katmanında yapılabilir.

H3 hücreleri için ayrı UUID üretilmez.

Hex'in doğal kimliği H3 index'tir.

4. PostgreSQL Schema Organizasyonu

MVP için tek PostgreSQL database içinde mantıksal schema kullanılabilir:

public veya app

İlk aşamada aşırı schema ayrımı gerekli değildir.

Önerilen basit yapı:

```
app.players
app.pets
app.cities
app.city_boundaries
app.rule_sets
app.seasons
app.walks
app.walk_points
app.walk_hex_results
app.territory_hexes
app.player_season_scores
app.player_lifetime_stats
app.outbox_events
```

Küçük ekip ve MVP için yalnızca `public` schema kullanmak da mümkündür.

5. Enum Yaklaşımı

PostgreSQL native enum yerine MVP'de çoğunlukla:

```
varchar / text
+
CHECK constraint
```

kullanılması önerilir.

Örnek:

```

status varchar(24) NOT NULL
CHECK (
  status IN (
    'CREATED',
    'ACTIVE',
    'SUBMITTED',
    'PROCESSING',
    'RESOLVED',
    'REJECTED',
    'FAILED'
  )
)

```

Sebebi:

- Native enum migration'ları daha katı olabilir.
- Yeni state eklemek daha kontrollü fakat daha zor olabilir.
- Text + CHECK uygulama sürümleri arasında daha esnek yönetilir.

Sık sorgulanan değerlerde text yerine `smallint` kullanılabilir; ancak okunabilirlik kaybı oluşur.

MVP'de okunabilirlik önceliklidir.

6. players

6.1 Amaç

Oyuncunun oyun içi profilini saklar.

Authentication hesabıyla oyun profilini ayırmak uzun vadede daha temizdir.

```

CREATE TABLE players (
  id                uuid PRIMARY KEY,
  auth_subject      varchar(191) NOT NULL,
  display_name      varchar(40) NOT NULL,
  home_city_id      uuid NULL,
  status            varchar(24) NOT NULL DEFAULT 'ACTIVE',

  created_at        timestampz NOT NULL,
  updated_at        timestampz NOT NULL,
  deleted_at        timestampz NULL,

  CONSTRAINT uq_players_auth_subject
    UNIQUE (auth_subject),

  CONSTRAINT ck_players_status

```

```
);  
CHECK (status IN ('ACTIVE', 'SUSPENDED', 'DELETED'))
```

6.2 Kolon Açıklamaları

auth_subject

Authentication provider'dan gelen kalıcı kimlik.

Email'in oyuncunun ana kimliği olarak kullanılmaması önerilir.

Email değişebilir.

display_name

Case-sensitive görünüm değeri.

Benzersiz username istenirse ayrı kolon kullanılmalıdır:

```
username  
username_normalized
```

MVP'de username uniqueness zorunlu değilse eklenmemelidir.

deleted_at

Soft delete için kullanılabilir.

Oyuncu silindiğinde bütün ilişkisel veriler anında fiziksel olarak silinmeyebilir.

Gizlilik ve retention politikası ayrıca tanımlanmalıdır.

6.3 İndeksler

```
CREATE INDEX idx_players_home_city_active  
ON players (home_city_id)  
WHERE status = 'ACTIVE';
```

PostgreSQL partial index, yalnızca belirli predicate'i karşılayan satırları indekslemeye izin verir. Bu, aktif kayıtları hedefleyen küçük ve verimli indeksler oluşturmak için kullanılabilir.

7. pets

```
CREATE TABLE pets (  
  id                uuid PRIMARY KEY,  
  owner_id          uuid NOT NULL,  
  name              varchar(40) NOT NULL,  
  status            varchar(24) NOT NULL DEFAULT 'ACTIVE',  
  
  created_at        timestampz NOT NULL,  
  updated_at        timestampz NOT NULL,  
  deleted_at        timestampz NULL,  
  
  CONSTRAINT fk_pets_owner  
    FOREIGN KEY (owner_id)  
    REFERENCES players(id),  
  
  CONSTRAINT ck_pets_status  
    CHECK (status IN ('ACTIVE', 'INACTIVE', 'DELETED'))  
);
```

İndeks:

```
CREATE INDEX idx_pets_owner_active  
ON pets (owner_id)  
WHERE status = 'ACTIVE';
```

MVP'de bir oyuncunun tek aktif pet'e sahip olması istenirse:

```
CREATE UNIQUE INDEX uq_pets_one_active_per_owner  
ON pets (owner_id)  
WHERE status = 'ACTIVE';
```

Bu kural ürün kararı kesinleşirse eklenmelidir.

8. cities

```
CREATE TABLE cities (  
  id                uuid PRIMARY KEY,  
  code              varchar(50) NOT NULL,  
  name              varchar(100) NOT NULL,  
  country_code      char(2) NOT NULL,  
  timezone          varchar(64) NOT NULL,  
  status            varchar(24) NOT NULL,  
  active_rule_set_id uuid NULL,
```



```

created_at          timestampz NOT NULL,
updated_at          timestampz NOT NULL,

CONSTRAINT uq_cities_code
    UNIQUE (code),

CONSTRAINT ck_cities_status
    CHECK (status IN ('ACTIVE', 'COMING_SOON', 'FROZEN'))
);

```

`timezone` IANA timezone adı olarak tutulmalıdır:

```

Europe/Istanbul
America/Denver
America/Los_Angeles

```

UTC offset tutulmamalıdır.

Offset daylight saving nedeniyle değişebilir.

9. city_boundaries

Şehir sınırlarının versiyonlanması gerekir.

Polygon zaman içinde değişebilir veya veri kaynağı güncellenebilir.

```

CREATE TABLE city_boundaries (
    id                uuid PRIMARY KEY,
    city_id            uuid NOT NULL,
    version            integer NOT NULL,
    boundary           geometry(MultiPolygon, 4326) NOT NULL,
    is_active          boolean NOT NULL DEFAULT false,

    source             varchar(100) NULL,
    created_at         timestampz NOT NULL,

    CONSTRAINT fk_city_boundaries_city
        FOREIGN KEY (city_id)
        REFERENCES cities(id),

    CONSTRAINT uq_city_boundary_version
        UNIQUE (city_id, version)
);

```

Spatial index:

```
CREATE INDEX idx_city_boundaries_geometry
ON city_boundaries
USING GIST (boundary);
```

PostGIS spatial sorgularda GiST tabanlı R-tree indeksleri kullanır.

Aktif boundary için:

```
CREATE UNIQUE INDEX uq_city_boundaries_one_active
ON city_boundaries (city_id)
WHERE is_active = true;
```

10. rule_sets

RuleSet aktif olduktan sonra immutable olmalıdır.

```
CREATE TABLE rule_sets (
  id                uuid PRIMARY KEY,
  version           varchar(50) NOT NULL,
  status            varchar(24) NOT NULL,

  rules             jsonb NOT NULL,

  checksum          varchar(64) NOT NULL,
  created_at        timestampz NOT NULL,
  activated_at      timestampz NULL,
  retired_at        timestampz NULL,

  CONSTRAINT uq_rule_sets_version
    UNIQUE (version),

  CONSTRAINT uq_rule_sets_checksum
    UNIQUE (checksum),

  CONSTRAINT ck_rule_sets_status
    CHECK (status IN ('DRAFT', 'ACTIVE', 'RETIRED')),

  CONSTRAINT ck_rule_sets_rules_object
    CHECK (jsonb_typeof(rules) = 'object')
);
```

PostgreSQL `jsonb`, JSON dokümanları üzerinde sorgulama ve indexing seçenekleri sunar.

10.1 Neden JSONB?

RuleSet:

- Nested yapıdadır.
- Sürümden sürüme yeni parametreler alabilir.
- Çok yoğun relation sorgularına konu olmaz.
- Tek parça olarak engine'e yüklenir.
- Immutable tutulur.

Bu nedenle JSONB mantıklıdır.

Ancak şu alanlar kolon olarak ayrılmıştır:

```
id
version
status
checksum
activated_at
```

Çünkü bunlarla sık sorgu ve constraint çalışır.

10.2 Rules Örneği

```
{
  "walk": {
    "minDurationSeconds": 300,
    "minDistanceMeters": 300,
    "maxSpeedMps": 4.5,
    "maxAccuracyMeters": 40
  },
  "movement": {
    "h3Resolution": 9,
    "minHexPresenceSeconds": 8,
    "minHexDistanceMeters": 15
  },
  "territory": {
    "maxDominance": 100,
    "initialDominance": 60,
    "ownerVisitGain": 10,
    "enemyAttackDamage": 15,
    "dailyDecay": 5
  }
}
```

RuleSet'in her alanını ayrı tablo ve kolonlara bölmek gereksiz normalizasyon olur.

11. seasons

```
CREATE TABLE seasons (  
  id                uuid PRIMARY KEY,  
  season_number     integer NOT NULL,  
  name              varchar(100) NULL,  
  status            varchar(24) NOT NULL,  
  
  starts_at         timestampz NOT NULL,  
  ends_at           timestampz NOT NULL,  
  
  rule_set_id       uuid NOT NULL,  
  
  created_at        timestampz NOT NULL,  
  closed_at         timestampz NULL,  
  
  CONSTRAINT uq_seasons_number  
    UNIQUE (season_number),  
  
  CONSTRAINT fk_seasons_rule_set  
    FOREIGN KEY (rule_set_id)  
    REFERENCES rule_sets(id),  
  
  CONSTRAINT ck_seasons_status  
    CHECK (  
      status IN (  
        'SCHEDULED',  
        'ACTIVE',  
        'CLOSED',  
        'ARCHIVED'  
      )  
    ),  
  
  CONSTRAINT ck_seasons_dates  
    CHECK (ends_at > starts_at)  
);
```

Aktif sezon:

```
CREATE UNIQUE INDEX uq_seasons_one_active  
ON seasons ((true))  
WHERE status = 'ACTIVE';
```

Bu global tek sezon varsayımı içindir.

Şehir bazlı sezon gelirse `city_id` eklenerek unique constraint değiştirilir.

12. walks

Walk tablosu lifecycle ve idempotency'nin merkezidir.

```
CREATE TABLE walks (  
  id                                uuid PRIMARY KEY,  
  
  player_id                        uuid NOT NULL,  
  pet_id                          uuid NOT NULL,  
  city_id                         uuid NOT NULL,  
  season_id                       uuid NOT NULL,  
  rule_set_id                     uuid NOT NULL,  
  
  status                          varchar(24) NOT NULL,  
  validation_status               varchar(24) NULL,  
  
  started_at                      timestampz NOT NULL,  
  ended_at                       timestampz NULL,  
  submitted_at                   timestampz NULL,  
  processing_started_at          timestampz NULL,  
  evaluated_at                   timestampz NULL,  
  resolved_at                    timestampz NULL,  
  
  total_distance_m                double precision NULL,  
  valid_distance_m                double precision NULL,  
  duration_seconds                integer NULL,  
  valid_route_ratio               real NULL,  
  
  engine_version                  varchar(50) NULL,  
  input_hash                      varchar(64) NULL,  
  
  rejection_reasons              jsonb NULL,  
  processing_attempts             smallint NOT NULL DEFAULT 0,  
  
  created_at                     timestampz NOT NULL,  
  updated_at                     timestampz NOT NULL,  
  
  CONSTRAINT fk_walks_player  
    FOREIGN KEY (player_id)  
    REFERENCES players(id),  
  
  CONSTRAINT fk_walks_pet  
    FOREIGN KEY (pet_id)  
    REFERENCES pets(id),  
  
  CONSTRAINT fk_walks_city  
    FOREIGN KEY (city_id)  
    REFERENCES cities(id),
```

```

CONSTRAINT fk_walks_season
    FOREIGN KEY (season_id)
    REFERENCES seasons(id),

CONSTRAINT fk_walks_rule_set
    FOREIGN KEY (rule_set_id)
    REFERENCES rule_sets(id),

CONSTRAINT ck_walks_status
    CHECK (
        status IN (
            'CREATED',
            'ACTIVE',
            'SUBMITTED',
            'PROCESSING',
            'RESOLVED',
            'REJECTED',
            'FAILED'
        )
    ),

CONSTRAINT ck_walks_validation_status
    CHECK (
        validation_status IS NULL
        OR validation_status IN (
            'VALID',
            'PARTIALLY_VALID',
            'INVALID'
        )
    ),

CONSTRAINT ck_walks_duration
    CHECK (
        duration_seconds IS NULL
        OR duration_seconds >= 0
    ),

CONSTRAINT ck_walks_route_ratio
    CHECK (
        valid_route_ratio IS NULL
        OR valid_route_ratio BETWEEN 0 AND 1
    )
);

```

12.1 Idempotency

```

CREATE UNIQUE INDEX uq_walks_player_input_hash
    ON walks (player_id, input_hash)
    WHERE input_hash IS NOT NULL;

```

Bu, aynı oyuncunun aynı route input'unu iki kez işleyerek skor üretmesini engellemeye yardımcı olur.

`walk_id` zaten primary key'dir.

API idempotency key ayrıca tutulabilir:

```
ALTER TABLE walks
ADD COLUMN idempotency_key varchar(100) NULL;
```

```
CREATE UNIQUE INDEX uq_walks_player_idempotency
ON walks (player_id, idempotency_key)
WHERE idempotency_key IS NOT NULL;
```

12.2 Bir Oyuncunun Tek Aktif Walk'ı

```
CREATE UNIQUE INDEX uq_walks_one_active_per_player
ON walks (player_id)
WHERE status IN ('CREATED', 'ACTIVE');
```

`SUBMITTED` veya `PROCESSING` yeni walk başlatmayı engellemeli mi, ürün davranışına göre karar verilir.

Benim önerim:

ACTIVE tek olmalı.
Önceki Walk processing durumundayken yeni Walk başlayabilir.

Böylece backend kuyruğu kullanıcı deneyimini bloklamaz.

12.3 Sorgu İndeksleri

```
CREATE INDEX idx_walks_player_created
ON walks (player_id, created_at DESC);
```

```
CREATE INDEX idx_walks_status_submitted
ON walks (status, submitted_at)
WHERE status IN ('SUBMITTED', 'PROCESSING');
```

```
CREATE INDEX idx_walks_city_resolved
ON walks (city_id, resolved_at DESC)
WHERE status = 'RESOLVED';
```

13. walk_points

Raw GPS noktaları yüksek hacimli tablodur.

```
CREATE TABLE walk_points (  
  walk_id          uuid NOT NULL,  
  sequence_no      integer NOT NULL,  
  
  recorded_at      timestampz NOT NULL,  
  location         geography(Point, 4326) NOT NULL,  
  
  accuracy_m       real NOT NULL,  
  altitude_m       real NULL,  
  speed_mps        real NULL,  
  
  is_mock_location boolean NOT NULL DEFAULT false,  
  
  received_at      timestampz NOT NULL,  
  
  PRIMARY KEY (walk_id, sequence_no),  
  
  CONSTRAINT fk_walk_points_walk  
    FOREIGN KEY (walk_id)  
    REFERENCES walks(id)  
    ON DELETE CASCADE,  
  
  CONSTRAINT ck_walk_points_sequence  
    CHECK (sequence_no >= 0),  
  
  CONSTRAINT ck_walk_points_accuracy  
    CHECK (accuracy_m >= 0)  
);
```

13.1 Neden geography(Point)?

Gerçek dünya koordinatıdır.

Mesafe hesaplarında metre bazlı davranış daha doğaldır.

Ancak H3 conversion application/engine tarafında yapılabilir.

13.2 Neden Lat ve Lng Ayrı Kolon Değil?

Şu yapı gereksiz tekrar üretir:


```
latitude
longitude
location
```

Tek authoritative konum kolonu tercih edilmelidir:

```
location geography(Point, 4326)
```

Engine için veri okunurken:

```
ST_Y(location::geometry)
ST_X(location::geometry)
```

ile koordinatlar çıkarılabilir.

Performans testlerinde dönüşüm maliyeti sorun olursa generated veya ayrı kolon değerlendirilebilir; baştan kopyalanmamalıdır.

13.3 Partitioning

`walk_points`, büyüme ihtimali en yüksek tablodur.

Partition için iki aday:

```
RANGE (recorded_at)
HASH (walk_id)
```

MVP başlangıcında aylık range partition mantıklıdır:

```
walk_points_2026_07
walk_points_2026_08
```

Avantajlar:

- Retention kolaylaşır
- Eski partition drop edilebilir
- Vacuum ve index yönetimi kolaylaşır
- Büyük tablo operasyonları bölünür

Ancak partitioning ilk günden yalnızca operasyonel olarak yönetilebiliyorsa uygulanmalıdır.

Küçük pilotta önce normal tablo ile başlanıp hacim ölçülebilir.

PostgreSQL declarative partitioning desteği sunar; partition key'in unique constraint tasarımıyla birlikte düşünülmesi gerekir.

13.4 Spatial Index Gerekli mi?

Raw points üzerinde çoğu sorgu:

```
walk_id + sequence_no
```

ile yapılacaktır.

Bu nedenle ilk aşamada bütün `walk_points.location` için GiST index oluşturmak gereksiz olabilir.

Spatial index yalnızca şu sorgular gerçekten yapılacaksa eklenmelidir:

- Bir alandaki bütün raw noktalar
- Geo analytics
- Fraud heatmap
- Spatial audit

İndekslerin de yazma maliyeti vardır.

Sorgulanmayan kolon için index oluşturulmayacaktır.

14. walk_hex_results

Bir walk'ın hangi hex'leri geçerli şekilde etkilediğinin kalıcı özetidir.

```
CREATE TABLE walk_hex_results (  
  walk_id          uuid NOT NULL,  
  hex_id           bigint NOT NULL,  
  
  presence_seconds integer NOT NULL,  
  distance_m       real NOT NULL,  
  entry_count      smallint NOT NULL,  
  
  action_type      varchar(32) NOT NULL,  
  
  dominance_before smallint NULL,  
  dominance_after  smallint NULL,  
  
  previous_owner_id uuid NULL,  
  new_owner_id      uuid NULL,  
  
  created_at       timestampz NOT NULL,  
  
  PRIMARY KEY (walk_id, hex_id),  
  
  CONSTRAINT fk_walk_hex_results_walk
```

```

FOREIGN KEY (walk_id)
REFERENCES walks(id)
ON DELETE CASCADE,

CONSTRAINT ck_walk_hex_action
CHECK (
    action_type IN (
        'NO_CHANGE',
        'EMPTY_CAPTURE',
        'OWNER_DEFENSE',
        'ENEMY_ATTACK',
        'OWNERSHIP_TRANSFER'
    )
)
);

```

14.1 Neden Saklıyoruz?

Bu tablo:

- Walk sonucu açıklaması
- Kullanıcı itirazları
- Replay karşılaştırması
- Simülasyon analizi
- Debugging
- Idempotency audit

için değerlidir.

Ancak authoritative territory state değildir.

Authoritative state:

```
territory_hexes
```

15. H3 Veri Tipi

H3 cell index 64-bit yapı içinde paketlenir.

PostgreSQL'de iki ana seçenek vardır:

```
bigint
varchar(16)
```

15.1 Öneri: bigint

```
hex_id bigint
```

Avantajları:

- 8 byte
- Daha küçük index
- Daha hızlı karşılaştırma
- B-tree için uygun
- Network payload'da string'e çevrilebilir

Dikkat:

H3 index kavramsal olarak unsigned 64-bit'tir.

PostgreSQL `bigint` signed 64-bit'tir.

Kullanılacak H3 binding'in değer dönüşümü dikkatle test edilmelidir.

Alternatif:

```
hex_id text
```

daha güvenli ve okunaklıdır ancak daha fazla storage/index alanı kullanır.

Karar kuralı:

Go H3 binding ile PostgreSQL signed bigint round-trip güvenli doğrulanırsa bigint kullanılacaktır. Aksi halde canonical H3 string kullanılacaktır.

Domain tipi değişmez:

```
type HexID uint64
```

Persistence mapper dönüşümü yönetir.

16. territory_hexes

Oyunun en kritik tablosudur.

```
CREATE TABLE territory_hexes (  
  city_id          uuid NOT NULL,  
  hex_id          bigint NOT NULL,
```

```

owner_id          uuid NULL,
dominance          smallint NOT NULL,

last_updated_at   timestampz NOT NULL,
last_visited_at   timestampz NULL,

version           bigint NOT NULL DEFAULT 1,

PRIMARY KEY (city_id, hex_id),

CONSTRAINT fk_territory_hex_city
    FOREIGN KEY (city_id)
    REFERENCES cities(id),

CONSTRAINT fk_territory_hex_owner
    FOREIGN KEY (owner_id)
    REFERENCES players(id),

CONSTRAINT ck_territory_dominance
    CHECK (dominance BETWEEN 0 AND 100),

CONSTRAINT ck_territory_owner_dominance
    CHECK (
        (owner_id IS NULL AND dominance = 0)
        OR
        (owner_id IS NOT NULL AND dominance >= 1)
    ),

CONSTRAINT ck_territory_version
    CHECK (version > 0)
);

```

16.1 Primary Key

```
(city_id, hex_id)
```

Neden yalnızca `hex_id` değil?

H3 ID global olarak benzersiz kabul edilebilir; ancak `city_id`:

- City-scoped sorguları kolaylaştırır
- Data locality sağlar
- İleride partition/sharding için alan açar
- Aynı hex'in hangi aktif şehir kapsamında olduğunu açıklar

Teknik olarak aynı H3 hücresinin iki şehirde bulunması boundary çakışmasıdır ve uygulama tarafından engellenmelidir.

16.2 Dominance Constraint

MVP'de dominance maksimumu RuleSet'ten gelir.

Database CHECK içinde sabit `100` kullanmak uzun vadede esnekliği azaltır.

İki seçenek vardır:

Seçenek A — Sabit teknik aralık

```
CHECK (dominance BETWEEN 0 AND 32767)
```

Business maksimumunu engine kontrol eder.

Seçenek B — MVP maksimumu 100 constraint'i

```
CHECK (dominance BETWEEN 0 AND 100)
```

RuleSet maksimumu ileride değişirse migration gerekir.

Benim önerim:

```
CHECK (dominance BETWEEN 0 AND 10000)
```

Database teknik güvenlik sınırı koyar.

Engine RuleSet'in gerçek maksimumunu uygular.

Böylece RuleSet değişimi migration gerektirmez.

16.3 Optimistic Locking

Update:

```
UPDATE territory_hexes
SET
  owner_id = $new_owner_id,
  dominance = $new_dominance,
  last_updated_at = $evaluated_at,
  last_visited_at = $visited_at,
  version = version + 1
WHERE city_id = $city_id
AND hex_id = $hex_id
AND version = $expected_version;
```

Affected row:

```
1 → başarı
0 → conflict
```

Bu, aynı hex'e eşzamanlı saldırıların sessiz veri kaybına neden olmasını engeller.

16.4 Empty Hex Insert

Sahipsiz bütün dünya hex'lerini önceden oluşturmayacağız.

İlk capture sırasında:

```
INSERT INTO territory_hexes (...)
VALUES (...)
ON CONFLICT (...) ...
```

kullanılır.

Yalnızca oyun tarafından etkilenmiş hex'ler saklanır.

Bu çok önemli bir storage tasarrufudur.

16.5 İndeksler

Owner sorguları:

```
CREATE INDEX idx_territory_owner_city
ON territory_hexes (owner_id, city_id)
WHERE owner_id IS NOT NULL;
```

Harita sorguları için H3 parent kolonunun gerekmesi mümkündür.

Örnek:

```
parent_hex_res_7
```

Ancak final resolution kesinleşmeden ve query pattern ölçülmeden bu kolon eklenmemelidir.

İleride generated column veya application tarafından doldurulan kolon kullanılabilir. PostgreSQL generated columns, diğer kolonlardan hesaplanan değerleri sanal veya saklanan kolon olarak tanımlayabilir.

17. Territory Geometrisi Saklanmalı mı?

Her `territory_hexes` satırında polygon saklamak önerilmez.

Yanlış:

```
hex_id
hex_polygon
```

H3 polygon'u hex ID'den üretilebilir.

Her satırda polygon saklamak:

- Gereksiz kopya veri
- Daha yüksek storage
- Daha ağır index
- Resolution değişiminde karmaşa

yaratır.

Harita çizimi için:

```
Hex ID → Flutter / backend H3 boundary
```

kullanılabilir.

Şehir polygon'u PostGIS'te saklanır.

Territory hex polygon'ları varsayılan olarak saklanmaz.

18. player_season_scores

```
CREATE TABLE player_season_scores (  
  player_id          uuid NOT NULL,  
  season_id          uuid NOT NULL,  
  city_id            uuid NOT NULL,  
  
  active_hex_count   integer NOT NULL DEFAULT 0,  
  capture_count      integer NOT NULL DEFAULT 0,  
  steal_count        integer NOT NULL DEFAULT 0,  
  defense_count      integer NOT NULL DEFAULT 0,  
  
  score_reached_at   timestampz NULL,  
  
  version            bigint NOT NULL DEFAULT 1,
```



```

updated_at          timestampz NOT NULL,

PRIMARY KEY (player_id, season_id, city_id),

CONSTRAINT fk_player_season_score_player
    FOREIGN KEY (player_id)
    REFERENCES players(id),

CONSTRAINT fk_player_season_score_season
    FOREIGN KEY (season_id)
    REFERENCES seasons(id),

CONSTRAINT fk_player_season_score_city
    FOREIGN KEY (city_id)
    REFERENCES cities(id),

CONSTRAINT ck_player_season_score_nonnegative
    CHECK (
        active_hex_count >= 0
        AND capture_count >= 0
        AND steal_count >= 0
        AND defense_count >= 0
    )
);

```

18.1 Neden city_id?

Oyuncu birden fazla aktif şehirde oynarsa skorlar şehir bazlı ayrılır.

MVP'de tek home city olsa bile bu kolon ileride migration ihtiyacını azaltır ve leaderboard sorgusunu kolaylaştırır.

18.2 Atomic Increment

```

INSERT INTO player_season_scores (...)
VALUES (...)
ON CONFLICT (player_id, season_id, city_id)
DO UPDATE SET
    active_hex_count =
        player_season_scores.active_hex_count
        + EXCLUDED.active_hex_count,

    capture_count =
        player_season_scores.capture_count
        + EXCLUDED.capture_count,

    steal_count =
        player_season_scores.steal_count
        + EXCLUDED.steal_count,

```

```

defense_count =
    player_season_scores.defense_count
    + EXCLUDED.defense_count,

version = player_season_scores.version + 1,
updated_at = EXCLUDED.updated_at;

```

Negatif active hex sonucunu engellemek için:

- Application delta'yı doğrular
- Database CHECK constraint korur
- Anomaly durumunda transaction rollback olur

18.3 Leaderboard İndeksi

```

CREATE INDEX idx_season_leaderboard
ON player_season_scores (
    city_id,
    season_id,
    active_hex_count DESC,
    steal_count DESC,
    defense_count DESC,
    score_reached_at ASC,
    player_id
);

```

B-tree index sıralı sonuç üretimine yardımcı olabilir ve uygun sorgularda ayrı sort ihtiyacını azaltabilir.

Bu index aynı zamanda covering index olarak tasarlanabilir; PostgreSQL bazı sorguları heap erişimi olmadan index-only scan ile cevaplayabilir.

19. player_lifetime_stats

Lifetime değerleri season satırlarında tekrar tekrar tutulmamalıdır.

```

CREATE TABLE player_lifetime_stats (
    player_id          uuid PRIMARY KEY,

    capture_count      bigint NOT NULL DEFAULT 0,
    steal_count        bigint NOT NULL DEFAULT 0,
    valid_walk_count   bigint NOT NULL DEFAULT 0,
    total_distance_m   bigint NOT NULL DEFAULT 0,
    alpha_win_count    integer NOT NULL DEFAULT 0,

    version            bigint NOT NULL DEFAULT 1,

```

```

updated_at          timestampz NOT NULL,

CONSTRAINT fk_lifetime_stats_player
    FOREIGN KEY (player_id)
    REFERENCES players(id),

CONSTRAINT ck_lifetime_stats_nonnegative
    CHECK (
        capture_count >= 0
        AND steal_count >= 0
        AND valid_walk_count >= 0
        AND total_distance_m >= 0
        AND alpha_win_count >= 0
    )
);

```

Böylece:

```

Weekly state → player_season_scores
Lifetime state → player_lifetime_stats

```

net ayrılır.

20. processed_score_changes

Walk idempotency yalnızca Walk status'a bağlı bırakılmamalıdır.

Skor delta'larının iki kez uygulanmasını engellemek için ledger benzeri kayıt önerilir.

```

CREATE TABLE processed_score_changes (
    walk_id          uuid NOT NULL,
    player_id        uuid NOT NULL,
    metric            varchar(40) NOT NULL,
    reason            varchar(40) NOT NULL,
    delta             bigint NOT NULL,
    created_at        timestampz NOT NULL,

    PRIMARY KEY (
        walk_id,
        player_id,
        metric,
        reason
    )
);

```

Bu tablo sayesinde aynı Walk sonucu tekrar uygulanırsa bile aynı skor değişikliği ikinci kez yazılamaz.

MVP’de bu güvenlik katmanı değerlidir.

21. territory_change_log

Authoritative state yalnızca son durumu tutar.

Audit için append-only history tutulmalıdır.

```
CREATE TABLE territory_change_log (  
  id                uuid PRIMARY KEY,  
  
  walk_id           uuid NOT NULL,  
  city_id           uuid NOT NULL,  
  hex_id            bigint NOT NULL,  
  
  change_type       varchar(32) NOT NULL,  
  
  previous_owner_id uuid NULL,  
  new_owner_id      uuid NULL,  
  
  dominance_before  smallint NOT NULL,  
  dominance_after   smallint NOT NULL,  
  
  previous_version  bigint NOT NULL,  
  new_version       bigint NOT NULL,  
  
  occurred_at       timestampz NOT NULL,  
  created_at        timestampz NOT NULL,  
  
  CONSTRAINT fk_territory_change_walk  
    FOREIGN KEY (walk_id)  
    REFERENCES walks(id)  
);
```

Bu tablo:

- Kullanıcı itirazları
- Fraud analizi
- Replay
- Debugging
- Score reconciliation

için kullanılır.

Bu event sourcing değildir.

Authoritative current state yine:

territory_hexes

22. outbox_events

Transaction tamamlandıktan sonra notification, analytics ve cache işlemleri için kullanılır.

```
CREATE TABLE outbox_events (  
  id                                uuid PRIMARY KEY,  
  
  event_type                        varchar(80) NOT NULL,  
  aggregate_type                   varchar(50) NOT NULL,  
  aggregate_id                     varchar(100) NOT NULL,  
  
  payload                           jsonb NOT NULL,  
  
  occurred_at                      timestampz NOT NULL,  
  created_at                       timestampz NOT NULL,  
  
  available_at                     timestampz NOT NULL,  
  processed_at                     timestampz NULL,  
  
  attempt_count                    smallint NOT NULL DEFAULT 0,  
  last_error                       text NULL,  
  
  locked_at                        timestampz NULL,  
  locked_by                        varchar(100) NULL,  
  
  CONSTRAINT ck_outbox_attempt_count  
    CHECK (attempt_count >= 0)  
);
```

Worker sorgusu:

```
CREATE INDEX idx_outbox_pending  
  ON outbox_events (available_at, created_at)  
  WHERE processed_at IS NULL;
```

Worker'lar:

```
FOR UPDATE SKIP LOCKED
```

yaklaşımıyla paralel event alabilir.

Outbox ve Walk/territory değişiklikleri aynı transaction içinde yazılır.

Böylece:

```
DB commit oldu ama event kayboldu
```

problemi önlenir.

23. API Idempotency Tablosu Gerekli mi?

Walk için idempotency `walks` içinde tutulabilir.

Genel API çağrıları için ayrı tablo ancak ihtiyaç oluşursa eklenir:

```
api_idempotency_keys
```

MVP'de gereksiz genel altyapı kurulmayacaktır.

Walk upload ve finish endpoint'leri için:

```
player_id + idempotency_key
```

yeterlidir.

24. Harita Okuma Stratejisi

Harita endpoint'i bütün şehir verisini getirmez.

Temel sorgu kapsamı:

```
City  
Viewport / H3 parent cells  
Resolution  
Updated after timestamp
```

24.1 H3 Parent Read Model

Final H3 resolution örneğin 9 ise harita viewport sorgusunu hızlandırmak için parent resolution kolonları değerlendirilebilir:

```
parent_res_5  
parent_res_6  
parent_res_7
```

Ancak hepsini baştan eklemek gereksizdir.

Önerilen MVP:

```
city_id + hex_id
```

ile başlanır.

Viewport'taki H3 cell listesi backend'de hesaplanır:

```
SELECT  
  hex_id,  
  owner_id,  
  dominance,  
  last_updated_at  
FROM territory_hexes  
WHERE city_id = $1  
AND hex_id = ANY($2);
```

Bu yaklaşım:

- Spatial polygon index gerektirmez
- H3 primary key üzerinden hızlı lookup yapar
- İstemcinin yalnızca görünür alanı almasını sağlar

Viewport çok genişse H3 parent grouping eklenebilir.

H3 hiyerarşik yapısı parent ve child hücreler arasında geçiş yapılmasına izin verir.

25. Raw GPS Retention

Raw GPS kişisel ve hassas konum verisidir.

Sınırsız saklanmamalıdır.

Önerilen ilk politika:

```
Raw walk_points: 30-90 gün  
Walk summary: uzun süre
```

```
Walk hex result: uzun süre veya belirli retention
Territory change log: uzun süre / arşiv
```

Kesin süre hukuk ve ürün politikasıyla birlikte belirlenmelidir.

Retention sonrası:

- Raw noktalar silinebilir
- Sıkıştırılmış route object storage'a taşınabilir
- Yalnızca özet metrikler kalabilir

DB tasarımı silmeye izin vermelidir.

26. Object Storage Kullanımı

MongoDB yerine raw route arşivi için object storage düşünülebilir.

Örnek format:

```
walk-routes/{year}/{month}/{walk_id}.json.gz
```

veya:

```
protobuf / parquet / compressed binary
```

Database'de:

```
route_archive_uri
route_checksum
route_archived_at
```

tutulabilir.

Ancak Faz 1'de buna gerek olmayabilir.

Önce PostgreSQL route hacmi ölçülmelidir.

27. Redis Kullanım Alanları

Redis source of truth olmayacaktır.

Uygun alanlar:

- Aktif RuleSet cache
- Harita response cache
- Leaderboard cache
- Rate limit
- Kısa süreli distributed lock
- Processing job coordination
- Presence/online state

Redis'te tutulmaması gereken authoritative state:

- Hex owner
- Dominance
- Walk resolved sonucu
- Lifetime score
- RuleSet ana kaydı

Redis kaybolduğunda sistem PostgreSQL'den yeniden kurulabilmelidir.

28. Transaction Tasarımı

Bir Walk resolve işlemi için önerilen transaction:

```
BEGIN
```

1. Walk status kontrolü
2. Walk → PROCESSING veya mevcut processing ownership kontrolü
3. İlgili territory hex state'lerini oku
4. Engine sonucu uygula
5. Optimistic version update'leri
6. Territory change log insert
7. Score delta insert ve atomic apply
8. Walk hex result insert
9. Walk → RESOLVED / REJECTED
10. Outbox event insert

```
COMMIT
```

Ancak engine hesaplamasının tamamı uzun transaction içinde yapılmamalıdır.

Daha doğru akış:

1. Input/state oku
2. Transaction dışında engine hesapla
3. Kısa transaction aç
4. Expected version ile uygula

5. Conflict varsa rollback
6. Güncel state ile sınırlı retry

Bu, lock süresini azaltır.

29. Deadlock Önleme

Bir Walk birden fazla hex günceller.

İki Walk aynı hex setini farklı sırada kilitlerse deadlock riski oluşur.

Kural:

Hex güncellemeleri her zaman aynı deterministik sırada uygulanmalıdır.

Örnek:

```
ORDER BY hex_id ASC
```

Bütün transaction'lar aynı sırayı kullanır.

Ayrıca:

- Transaction kısa tutulur
- External API transaction içinde çağrılmaz
- Retry mekanizması bulunur
- Lock timeout belirlenir

30. Conflict Retry

Optimistic locking conflict durumunda:

```
Maximum retry: 2 veya 3
```

Önerilen:

1. State yeniden oku
2. Engine'in etkilenen kısmını yeniden hesapla
3. Kısa random jitter ile retry
4. Limit aşıldıysa RETRYABLE_CONFLICT

Client'ın bütün route'u tekrar göndermesine gerek yoktur.

Application mevcut Walk input'u üzerinden tekrar işlem yapabilir.

31. Partition Stratejisi

Her tablo partition edilmemelidir.

Partition adayları:

```
walk_points
territory_change_log
outbox_events
```

Partition gerekmeyen tablolar:

```
players
pets
cities
rule_sets
seasons
territory_hexes
player_lifetime_stats
```

`walks` büyürse tarih bazlı partition değerlendirilebilir.

İlk pilotta partition eklemenin operasyon maliyeti faydasından büyükse ertelenebilir.

Partition kararını satır sayısı değil şu faktörler belirler:

- Query pattern
- Retention
- Vacuum süresi
- Index boyutu
- Backup/restore
- Delete maliyeti

32. Index Tasarım Kuralları

32.1 Her Foreign Key Otomatik İndekslenmez

PostgreSQL foreign key tanımlandığında referencing kolon için otomatik index oluşturmaz.

Sorgu ve delete/update pattern'ine göre index eklenmelidir.

32.2 Fazla Index Yazmayı Yavaşlatır

Özellikle:

```
walk_points
territory_hexes
outbox_events
```

üzerinde yalnızca gerçek sorguları destekleyen indeksler bulunmalıdır.

32.3 Composite Index Kolon Sırası

Sorgu pattern'ine göre:

```
Equality columns
→ Range columns
→ Sort columns
```

değerlendirilir.

32.4 Index İsimleri

Standart:

```
pk_<table>
uq_<table>_<columns>
idx_<table>_<purpose>
fk_<table>_<relation>
ck_<table>_<rule>
```

33. Gereksiz Kopyaları Önleme

Aşağıdaki kopyalar yapılmayacaktır:

```
walks içinde player display name
walks içinde city name
territory_hexes içinde owner display name
player scores içinde pet name
her territory satırında H3 polygon
walk_points içinde hem lat/lng hem geometry
season score içinde lifetime totals
rule_sets içinde hem JSON hem bütün JSON alanlarının kolon kopyası
```

Read API için join maliyeti sorun olursa:

- Cache
- Materialized view
- Projection table
- Read replica

kullanılır.

Authoritative tablolar kirletilmez.

34. Read Model ve Materialized View

MVP'nin ilk gününde materialized view zorunlu değildir.

İleride kullanılabilecek örnekler:

```
city_leaderboard_current  
player_profile_summary  
city_territory_summary
```

Bunlar source of truth değildir.

Refresh:

- Event-driven projection
- Periyodik refresh
- Incremental update

ile yapılabilir.

Leaderboard için ilk aşamada doğru index'li `player_season_scores` yeterli olabilir.

35. Backup ve Recovery

Profesyonel database tasarımı yalnızca şema değildir.

Minimum gereksinimler:

```
Automated daily backup  
Point-in-time recovery  
WAL archiving  
Backup encryption  
Restore test  
Migration rollback plan
```

Production/staging ayrımı
Least-privilege DB users

Backup'ın var olması yeterli değildir.

Restore işlemi düzenli olarak test edilmelidir.

Önerilen recovery hedefleri ürün çıktığında tanımlanmalıdır:

RPO → Ne kadar veri kaybı kabul edilebilir?
RTO → Sistem ne kadar sürede geri açılmalı?

MVP pilot için örnek hedef:

RPO ≤ 5 dakika
RTO ≤ 60 dakika

Bu değerler hosting maliyetine göre kesinleştirilir.

36. Migration Stratejisi

Migration aracı olarak:

golang-migrate
Goose
Atlas

gibi bir araç seçilebilir.

Kural:

- Migration dosyaları repository'de tutulur.
- Migration manuel production SQL'iyle atlanmaz.
- Destructive migration iki aşamalı yapılır.
- Büyük tabloya blocking migration dikkatle uygulanır.
- Application eski ve yeni şemayla kısa süre uyumlu olabilir.

Örnek güvenli kolon kaldırma:

1. Kod eski kolonu kullanmayı bırakır.
2. Deploy edilir.
3. Veri bağımlılığı doğrulanır.
4. Sonraki migration'da kolon kaldırılır.

37. Database Roller

En az:

```
migration_user
application_user
readonly_user
worker_user
```

ayrımı düşünölmelidir.

Application user:

- Schema değıştirmekemeli
- Gereksiz table truncate yapamamalı
- Yalnızca gerekli CRUD izinlerine sahip olmalı

38. Production Connection Yönetimi

Go uygulaması connection pool kullanacaktır.

Dikkat:

```
Pod sayısı × MaxOpenConnections
```

PostgreSQL connection sınırını aşmamalıdır.

Gerekirse:

```
PgBouncer
```

kullanılabilir.

İlk aşamada küçük pool yeterlidir.

Connection sayısını yüksek tutmak doğrudan daha yüksek performans anlamına gelmez.

39. Önerilen MVP Tablo Listesi

Core

```
players
pets
cities
city_boundaries
rule_sets
seasons
walks
walk_points
territory_hexes
player_season_scores
player_lifetime_stats
```

Audit ve güvenilir işlem

```
walk_hex_results
territory_change_log
processed_score_changes
outbox_events
```

Toplam:

14 tablo

Bu sayı MVP için yüksek görünse de tabloların her biri ayrı ve gerçek bir sorumluluk taşır.

Gereksiz kopya tablo yoktur.

40. İlk MVP’de Ertelenebilecek Tablolar

Daha hızlı başlamak için şu iki tablo ertelenebilir:

```
processed_score_changes
territory_change_log
```

Ancak benim profesyonel önerim:

- `territory_change_log` kalsın

- `processed_score_changes` score idempotency transaction içinde başka bir unique mekanizmayla çözülebiliyorsa ertelenebilir

`walk_hex_results` ve `territory_change_log` kısmen benzer görünür.

Aralarındaki fark:

`walk_hex_results`

→ Bir yürüyüşün hesap sonucunu açıklar.

`territory_change_log`

→ Territory state'inde gerçekten uygulanmış değişikliği açıklar.

Conflict veya retry nedeniyle ikisi her zaman birebir aynı olmayabilir.

Bu nedenle ikisi de anlamlıdır.

41. MongoDB'nin Ne Zaman Eklenmesi Mantıklı Olur?

MongoDB yalnızca şu koşullardan biri oluşursa tekrar değerlendirilmelidir:

1. Raw route document'ları PostgreSQL üzerinde gerçek darboğaz yaratır.
2. Telemetry şeması çok hızlı değişir ve relational sorgu gerektirmez.
3. Ayrı ekip tarafından yönetilen bağımsız analytics workload oluşur.
4. MongoDB'nin document embedding modeli belirli bir API read pattern'ini belirgin şekilde hızlandırır.
5. Operasyon ekibi iki database'i güvenle yönetebilecek seviyeye gelir.

MongoDB şu amaçlarla eklenmemelidir:

"NoSQL daha hızlıdır."
"Join olmasın."
"İleride ölçeklenir."
"Raw JSON var."

Performans teknoloji isminden değil, doğru data modeli ve query pattern'den gelir.

42. Önerilen Nihai Veri Mimarisi

PostgreSQL + PostGIS

- |
- |— Authoritative relational state
- |— Territory ownership
- |— Walk lifecycle
- |— Scores
- |— Seasons
- |— RuleSets
- |— Spatial city boundaries
- |— Raw route data
- |— Transactional outbox

Redis

- |
- |— Cache
- |— Rate limiting
- |— Leaderboard cache
- |— Temporary coordination

Object Storage – ileride

- |
- |— Archived compressed raw routes

MongoDB

- |
- |— MVP’de yok

43. Kritik Invariant’ların Database Karşılığı

Domain kuralı	Database koruması
Bir walk iki kez işlenemez	Walk status + input hash unique
Oyuncunun bir aktif walk’ı olabilir	Partial unique index
Aynı hex tek current state taşır	Primary key (city_id, hex_id)
Concurrent update kaybolamaz	version optimistic lock
RuleSet değiştirilemez	Yeni version insert politikası
Score iki kez uygulanmamalı	Walk-based unique score ledger
Owner null ise dominance aktif olamaz	CHECK constraint
Event kaybolmamalı	Transactional outbox

Domain kuralı	Database koruması
Sezon skorları karışmamalı	Composite primary key
GPS point sırası tekrarlanamaz	(walk_id, sequence_no) primary key

44. Definition of Done

Database Design v1.0 şu koşulları sağlamalıdır:

1. PostgreSQL authoritative database olarak seçilmiş.
2. PostGIS yalnızca gerçek spatial ihtiyaçlarda kullanılmış.
3. MongoDB gereksiz yere eklenmemiş.
4. TerritoryHex tek source of truth.
5. H3 polygon'ları gereksiz kopyalanmamış.
6. Walk idempotency database seviyesinde korunmuş.
7. Aynı hex üzerindeki concurrent update tespit ediliyor.
8. Walk transaction'ı outbox event'leriyle birlikte commit oluyor.
9. Score state Player profilinden ayrılmış.
10. Weekly ve lifetime skorlar ayrılmış.
11. Raw GPS ana domain sorgularından ayrılmış.
12. Yüksek hacimli tablolar belirlenmiş.
13. Partition yalnızca gerekli tablolara planlanmış.
14. Leaderboard için uygun composite index bulunuyor.
15. Harita bütün şehir yerine viewport/H3 hücre listesiyle okunuyor.
16. RuleSet immutable ve versiyonlanmış.
17. Foreign key ve CHECK constraint'ler tanımlanmış.
18. Gereksiz kolon ve veri kopyaları kaldırılmış.
19. Backup, PITR ve restore test yaklaşımı tanımlanmış.
20. Database şeması engine ve API modellerinden ayrılmış.

45. Nihai Karar

MVP için önerilen veri altyapısı:

PostgreSQL + PostGIS

Raw route verileri de ilk aşamada PostgreSQL'de tutulacaktır.

MongoDB şimdilik eklenmeyecektir.

Veritabanının kalbi:

```
territory_hexes  
walks  
walk_points  
player_season_scores  
player_lifetime_stats  
outbox_events
```

olacaktır.

En kritik teknik kararlar:

```
Her hex bağımsız state.  
Her update version kontrollü.  
Her walk idempotent.  
Score değişimleri delta tabanlı.  
Outbox ana transaction'ın parçası.  
Raw GPS ayrı yüksek hacimli tablo.  
Harita H3 tabanlı toplu lookup.  
Leaderboard derived state.  
RuleSet immutable.
```

Bu yapı MVP'de basit kalır; aynı zamanda binlerce eşzamanlı kullanıcının farklı hex'lerde işlem yapmasına ve aynı hot hex üzerindeki conflict'lerin güvenli biçimde çözülmesine uygundur.